APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

KTU SPECIAL

We can together dream the B.tech

APJ ABDUL KALAM
TECHNOLOGICAL UNIVERSITY

• KTU STUDY MATERIALS	• SYLLABUS
• KTU LIVE NOTIFICATION	• SOLVED QUESTION PAPERS

JOIN WITH US

WWW.KTUSPECIAL.IN KTUSPECIAL t.me/ktuspecial1

PCITT302 – COA **MODULE-4**

Notes by Asst. Prof. Neomi Nelin Nicholas, IT Dept, TIST

Syllabus

Introduction to Parallel Processors: SISD, MIMD, SIMD, SPMD, and vector hardware multithreading, multicore and other shared memory multiprocessors, Case Study: Intel Core i7.

Introduction to Parallel Processors: SISD, MIMD, SIMD, SPMD, and vector hardware multithreading

The categorization of parallel hardware, which includes **SISD, MIMD, and SIMD**, was proposed in the 1960s and is based on the number of instruction streams and the number of data streams. **SPMD** describes the conventional programming approach used on MIMD systems, and **vector hardware** is an elegant interpretation of the SIMD architecture.

The architectural concepts derived from this model are described below, along with the related concept of hardware multithreading.

---

1. SISD (Single Instruction stream, Single Data stream)

SISD stands for **Single Instruction stream, Single Data stream**. This architecture describes a conventional **uniprocessor**. An example of a SISD architecture is the Intel Pentium 4.

2. MIMD (Multiple Instruction streams, Multiple Data streams)

MIMD stands for **Multiple Instruction streams, Multiple Data streams**. This architecture describes a conventional **multiprocessor**. An example of a MIMD architecture is the Intel Core i7.

While it is possible for different processors in a MIMD computer to run separate programs to coordinate toward a goal, programmers typically use the SPMD style.

3. SPMD (Single Program, Multiple Data streams)

SPMD stands for **Single Program, Multiple Data streams**. It is the **conventional programming model for MIMD** computers.

In this style, programmers normally write a **single program that runs on all processors** of the MIMD computer, using conditional statements when certain processors should execute different sections of code.

4. SIMD (Single Instruction stream, Multiple Data streams)

SIMD stands for **Single Instruction stream, Multiple Data streams**.

Core Characteristics:

• **Operation:** In SIMD, the **same instruction is applied to many data streams** simultaneously, as is seen in a vector processor.

• **Synchronization:** All parallel execution units are **synchronized** and respond to a **single instruction** that originates from a single program counter (PC).

• **Data Addressing:** Although they execute the same instruction, each execution unit has its own address registers, allowing each unit to operate on different data addresses.

• **Parallelism Type:** SIMD relies on a great deal of **identically structured data**, which is called **data-level parallelism**. Data-level parallelism is achieved by performing the same operation on independent data.

download from ktuspecial.in

• **Application:** SIMD works best when dealing with arrays within `for` loops. For instance, a single SIMD instruction might add 64 numbers by sending 64 data streams to 64 ALUs to form 64 sums within a single clock cycle.
• **Weakness:** SIMD is at its weakest in `case` or `switch` statements because execution units with the wrong data must be disabled. If there are n cases, SIMD processors may run at only $1/n$th of peak performance.
Implementations (Multimedia Extensions):
• The subword parallel instructions (like MMX, SSE, and AVX) are examples of SIMD. The middle letter of Intel's SSE acronym stands for SIMD.
• Advanced Vector Extensions (AVX) support the **simultaneous execution of four 64-bit floating-point numbers**.

5. Vector Hardware (An Interpretation of SIMD)

Vector architecture is described as an **older and more elegant interpretation of SIMD**. It is closely identified with computers designed by Seymour Cray starting in the 1970s and is well-matched to problems with high data-level parallelism.
Architectural Philosophy:
• **Data Flow:** The basic philosophy is to **collect data elements from memory**, put them in order into a large set of registers (**vector registers**), operate on them sequentially using pipelined execution units, and then write the results back to memory.
• **Parallel Execution:** Since vector elements are independent, they can be operated on in parallel, often using vector functional units with multiple parallel pipelines, called **vector lanes** (e.g., four lanes can complete four additions per cycle).

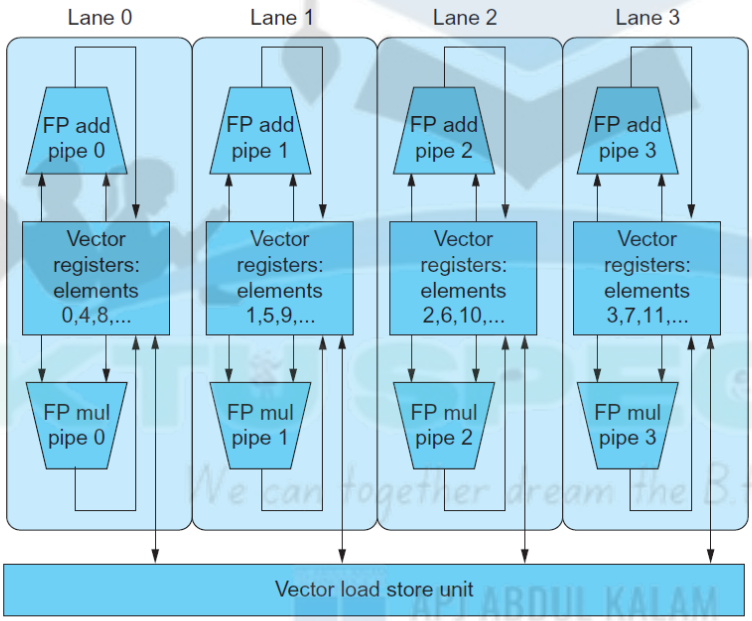

FIGURE 6.4 Structure of a vector unit containing four lanes. The vector-register storage is divided across the lanes, with each lane holding every fourth element of each vector register. The figure shows three vector functional units: an FP add, an FP multiply, and a load-store unit. Each of the vector arithmetic units contains four execution pipelines, one per lane, which acts in concert to complete a single vector instruction. Note how each section of the vector-register file only needs to provide enough read and write ports (see Chapter 4) for functional units local to its lane.

download from ktuspecial.in

Performance Advantages over Scalar/Multimedia Extensions:

• **Instruction Bandwidth:** A single vector instruction specifies a large amount of work, equivalent to an entire loop, which dramatically **reduces the instruction fetch and decode bandwidth** needed. This reduction in executed instructions also saves energy.

• **Hazard Management:** The hardware does not need to check for data hazards within a vector instruction because the programmer or compiler indicates the independence of results. Hardware only checks for hazards between two *vector instructions* once per vector operand, reducing pipeline stall frequency dramatically (e.g., 64 times lower in one example). Reduced checking saves both time and energy.

• **Control Hazards:** Because an entire loop is replaced by a predetermined vector instruction, **control hazards** (arising from the loop branch) are nonexistent.

• **Memory Efficiency:** Vector memory access patterns are known, and the cost of latency to main memory is incurred **only once for the entire vector**, not once for each word.

• **Flexible Access:** Unlike standard multimedia extensions, vector architectures support flexible data transfers, including **strided accesses** (loading every nth element) and **indexed accesses** (also called **gather-scatter**), where addresses are found in a vector register.

6. Hardware Multithreading

Hardware multithreading is a concept related to MIMD, especially from the programmer's perspective.

Definition and Mechanism:

• **Goal:** Hardware multithreading increases the utilization of a single processor by **switching to another thread when one thread is stalled**.

• **Sharing:** It allows **multiple threads to share the functional units of a single processor** in an overlapping fashion to efficiently utilize hardware resources.

• **Thread State:** To allow sharing, the processor must duplicate the independent state of each thread, requiring separate copies of the **register file and the program counter** for each thread.

• **Efficiency:** A thread switch must be much more efficient (potentially instantaneous) than a process switch (hundreds to thousands of processor cycles). A **thread** itself is a lightweight process that includes the program counter, register state, and stack, and threads commonly share a single address space.

Approaches to Hardware Multithreading:

Approach	Switching Mechanism	Advantages	Disadvantages
Fine-Grained Multithreading	Switches between threads **after every instruction**.	Can hide throughput losses from both short and long stalls.	Slows down the execution of individual threads, as a ready thread is delayed by instructions from other threads.
Coarse-Grained Multithreading	Switches threads only on **costly stalls** (e.g., a last-level cache miss).	Less likely to slow down the execution of an individual thread.	Limited in overcoming losses from shorter stalls due to **pipeline start-up costs**; the pipeline must be emptied/frozen, and the new thread must refill it.

download from ktuspecial.in

Simultaneous Multithreading (SMT)	Utilizes the resources of a multiple-issue, dynamically scheduled processor.	Exploits **thread-level parallelism** at the same time it exploits **instruction-level parallelism**. It is always executing instructions from multiple threads, using resources efficiently.	N/A (It is designed to lower the cost and improve utilization compared to traditional methods).

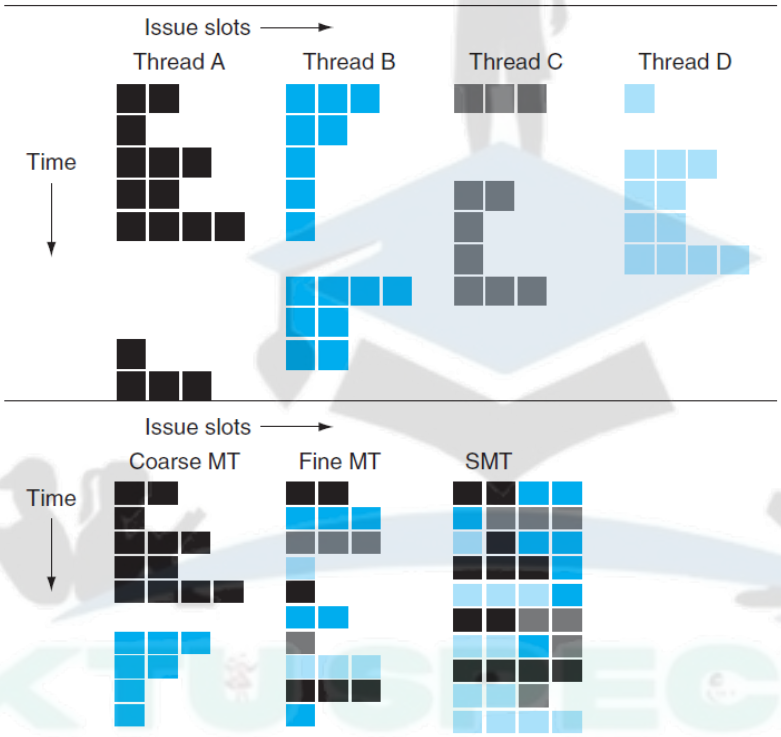

FIGURE 6.5 How four threads use the issue slots of a superscalar processor in different approaches. The four threads at the top show how each would execute running alone on a standard superscalar processor without multithreading support. The three examples at the bottom show how they would execute running together in three multithreading options. The horizontal dimension represents the instruction issue capability in each clock cycle. The vertical dimension represents a sequence of clock cycles. An empty (white) box indicates that the corresponding issue slot is unused in that clock cycle. The shades of gray and color correspond to four different threads in the multithreading processors. The additional pipeline start-up effects for coarse multithreading, which are not illustrated in this figure, would lead to further loss in throughput for coarse multithreading.

Explain multicore and other shared memory multiprocessors,

download from ktuspecial.in

The concept of multicore and other shared memory multiprocessors relates to the **MIMD (Multiple Instruction streams, Multiple Data streams)** architecture and is fundamentally defined by the use of a **single physical address space** shared among processors.

Here is an explanation of multicore and other shared memory multiprocessors based on the sources:

1. Shared Memory Multiprocessors (SMPs)

A shared memory multiprocessor (SMP) is defined as an architecture that offers the programmer a **single physical address space across all processors**.

- **Communication:** Processors communicate through **shared variables in memory**, and all processors can access any memory location using loads and stores.
- **Motivation:** This approach simplifies the task of parallel programming because programs do not need to concern themselves with where their data is located.
- **Addressing:** While "shared memory multiprocessor" is the standard term, a more accurate description would be "shared-address multiprocessor".
- **Operating System Support:** Even though they share a physical address space, these systems can still run independent jobs in their own virtual address spaces.

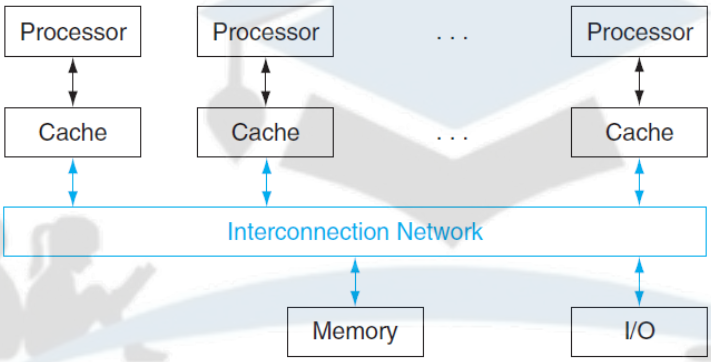

FIGURE 6.7 Classic organization of a shared memory multiprocessor.

2. Multicore Chips

Multicore chips are systems where multiple processors reside on a single chip, and they are nearly always implementations of **shared memory multiprocessors**.

- **MIMD Architecture:** Multicore computers represent the modern interpretation of the MIMD architecture, which stands for **Multiple Instruction streams, Multiple Data streams**. The Intel Core i7 is an example of a MIMD multicore processor.
- **Parallel Programming Challenge:** The major challenge of the last decade has been to efficiently program the increasing number of processors (cores) per chip to deliver on the performance potential of Moore's Law. Moore's Law is the observation that the number of

download from ktuspecial.in

transistors on a microchip doubles approximately every two years, leading to exponential increases in computing power and efficiency.

3. Styles of Single Address Space Multiprocessors

Shared memory multiprocessors are organized into two primary styles based on memory access latency:

1. **Uniform Memory Access (UMA):** In UMA multiprocessors, the latency (time) required to access a word in memory **does not depend on which processor asks for it**.
2. **Nonuniform Memory Access (NUMA):** In NUMA multiprocessors, **some memory accesses are much faster than others**, depending on which processor requests the word. This difference usually occurs because main memory is divided and attached to different microprocessors or different memory controllers on the same chip.
    - **Trade-offs:** While NUMA machines present harder programming challenges than UMA machines, they can scale to larger sizes and offer lower latency to nearby memory.

4. Data Consistency and Synchronization

When processors share an address space, mechanisms are required to ensure data integrity and coordination:

- **Cache Coherence:** When a common physical address space is used, the **hardware typically provides cache coherence** to ensure that all processors maintain a consistent view of the shared memory.
- **Synchronization:** Because processors operating in parallel normally share data, they need to coordinate their behavior. This process is called **synchronization**.
    - **Locks:** Synchronization is often implemented using a **lock**, a device that allows only one processor at a time to access shared data. Other processors must wait until the original processor unlocks the variable.

5. Programming Shared Memory Systems (OpenMP and SPMD)

The conventional way to program MIMD shared memory systems is the **Single Program, Multiple Data (SPMD)** style.

- **SPMD:** A single program runs on all processors, using conditional statements to guide different processors to different sections of the code.
- **OpenMP:** OpenMP is a popular example of a parallel programming system designed for shared memory multiprocessors.
    - **Function:** OpenMP is an **Application Programmer Interface (API)**, along with compiler directives and runtime routines, that provides a portable, scalable, and simple programming model.
    - **Goal:** Its primary goal is to **parallelize loops and perform reductions**. For instance, OpenMP uses pragmas (commands to the C macro preprocessor) to direct

download from ktuspecial.in

the compiler to spread loop work across threads. A macro preprocessor is a tool in programming that processes code before it is compiled. It performs text substitution and other preparatory tasks to make the code ready for the compiler. It is commonly used in languages like C and C++.

- **Example Reduction:** OpenMP includes commands to specify reduction operations, such as summing an array, leaving it to the OpenMP library to find efficient code to combine the partial sums using multiple processors.

Case Study of Intel Core i7 comparing it against a GPU architecture.

1. Architectural Classification and Multithreading

The Intel Core i7 is categorized based on the number of instruction streams and data streams:

• **MIMD (Multiple Instruction streams, Multiple Data streams):** The Intel Core i7 is listed as an example of an MIMD architecture. A conventional multiprocessor is defined as having multiple instruction streams and multiple data streams.

• **SIMD Extensions:** The Core i7 utilizes Streaming SIMD Extensions (SSE) instructions, which are a form of **SIMD** (Single Instruction stream, Multiple Data streams).

• **Hardware Multithreading:** The Intel Core i7 960 has hardware support for two threads per core. Simultaneous Multithreading (SMT) is a technique used to improve resource utilization on dynamically scheduled, out-of-order processors.

◦ Benchmarking on one core of an i7 processor using the PARSEC benchmarks showed an average speed-up from multithreading of **1.31**.

◦ The average improvement in energy efficiency achieved was **1.07**.

2. Benchmarking and Rooflines of the Intel Core i7 960

A specific case study was conducted by Intel researchers comparing the **quad-core Intel Core i7 960** (45-nanometer technology) with multimedia SIMD extensions against the NVIDIA Tesla GTX 280 GPU.

Intel Core i7 960 Specifications (Summary)

The key characteristics of the Core i7 960, purchased in Fall 2009, are:

Feature	Core i7-960 Value
Number of processing elements (cores)	4
Clock frequency	3.2 GHz
Technology	Intel 45 nm
Memory Bandwidth (measured)	16.4 GBytes/sec
Peak Single-precision SIMD FLOPS	102 GFLOP/Sec
Peak Double-precision SIMD FLOPS	51 GFLOP/sec

The Core i7 960 has a **double-precision ridge point of 3.1** (FLOP/byte) on the Roofline model.

Performance Insights from Kernel Benchmarking

The study compared the performance of several throughput computing kernels to understand the architectural differences between the Core i7 and the GTX 280.

download from ktuspecial.in

1. **Memory Bandwidth Limitations:** The GPU had 4.4x the memory bandwidth of the Core i7.

◦ For memory-intensive kernels with large working sets that purposely did not fit in the Core i7 cache (like LBM and SAXPY), the GPU ran 5.0x and 5.3x faster, respectively. This performance difference was explained by the memory bandwidth roofline slope.

2. **Compute Bandwidth:**

◦ For single-precision compute-bound kernels (SGEMM, Conv, FFT), the GTX 280 was 2.8x to 3.9x faster because its single-precision performance is 3 to 6 times faster than the Core i7.

◦ For double-precision kernels (MC), the GTX 280 was only 1.8x faster, corresponding to the GPU's double-precision performance being 1.5x faster than the i7.

3. **Instruction Support (Transcendentals):**

◦ The Bilat kernel, which uses transcendental functions, ran **5.7x faster** on the GTX 280 because the GPU supports these functions directly in hardware, whereas the Core i7 spent two-thirds of its time calculating them.

4. **Cache Benefits:** The caches and the effective use of **cache blocking** were highly important to the Core i7's performance.

◦ For Ray Casting (RC), cache blocking prevented the Core i7 from becoming memory bandwidth bound.

◦ Cache blocking allowed SGEMM, FFT, and SpMV to become compute bound on the Core i7.

◦ For the Sort kernel, the Core i7 actually ran **1.25x faster** than the GTX 280, even though the GPU generally excels at parallel data operations.

5. **Gather-Scatter Limitations:** This proved to be a major weakness for the Core i7's SIMD extensions.

◦ The Core i7's multimedia SIMD extensions (SSE/AVX) are designed for optimal performance when data accesses are contiguous or aligned on 16-byte boundaries.

◦ For the GJK kernel, which involves data scattered throughout main memory, the Core i7 received little benefit from SIMD.

◦ Because the GPU supported gather-scatter addressing (a feature found in vector architectures), the GTX 280 ran GJK a startling **15.2x faster** than the Core i7. Intel researchers noted that **6 of the 14 kernels** would better exploit SIMD if the Core i7 had more efficient gather-scatter support.

6. **Synchronization Performance:**

◦ The Core i7 benefited from hardware atomic instructions and a memory consistency model.

◦ For the Solv kernel, which requires frequent synchronization (barrier synchronization), the Core i7 ran **2.0x faster** than the GTX 280.

3. DGEMM Performance on Multi-Chip Core i7

A further case study detailed the performance acceleration of the DGEMM (matrix multiply) example using OpenMP on a dual-chip Intel Core i7 (Sandy Bridge generation) system, which provided a total of **16 cores**.

• By adding a single OpenMP pragma to the outermost loop, the work was spread across multiple threads.

• For the two largest matrices that did not fit entirely in the cache, the 16-threaded version achieved a **14x speedup** compared to the single-threaded version.

• This optimized DGEMM code operated at **174 GFLOPS** for 960 x 960 matrices.

download from ktuspecial.in

• Overall, the optimizations applied throughout the curriculum (Chapters 3 to 6) resulted in a total speedup of **over 200 times** compared to the original unoptimized C version (which ran at 0.8 GFLOPS).
• **Scaling Difficulty:** When running small 32 x 32 matrices that fit entirely within the first-level data cache, adding threads actually harmed performance; the 16-threaded version was almost half as fast as the single-threaded version.
• **Turbo Mode Impact:** When using 16 threads, the benefit of Turbo mode was reduced to a 1.11x average speedup, down from 1.27x for 1 or 2 threads, because less power budget was available for the increased number of active cores.

QUESTIONS

Differentiate between SISD, SIMD, and MIMD architectures with suitable examples.

The differentiation between **SISD, SIMD, and MIMD** architectures is based on a categorization proposed in the 1960s, which uses the number of instruction streams and the number of data streams.
---
1. SISD (Single Instruction stream, Single Data stream)
Definition and Characteristics:
• **Instruction/Data Streams:** SISD stands for **Single Instruction stream, Single Data stream**.
• **Description:** A SISD architecture is characteristic of a conventional **uniprocessor**.
• **Context:** This type of architecture is the conventional model where a single instruction operates on a single stream of data.
Suitable Example:
• An example of a SISD architecture is the **Intel Pentium 4**.
---
2. SIMD (Single Instruction stream, Multiple Data streams)
Definition and Characteristics:
• **Instruction/Data Streams:** SIMD stands for **Single Instruction stream, Multiple Data streams**.
• **Description:** In a SIMD architecture, the **same instruction is applied to many data streams** simultaneously.
• **Execution:** All parallel execution units in a SIMD system are synchronized and respond to a **single instruction** that originates from a **single program counter (PC)**. Although they execute the same instruction, each execution unit has its own address registers, allowing each unit to operate on different data addresses.
• **Application Focus:** SIMD works best when handling arrays within loops. It is highly effective when there is a great deal of identically structured data, which is known as **data-level parallelism** (parallelism achieved by performing the same operation on independent data).
• **Efficiency:** A single SIMD instruction can add many numbers (e.g., 64 numbers) by sending many data streams (e.g., 64 data streams) to execution units (e.g., 64 ALUs) to form multiple results (e.g., 64 sums) within a single clock cycle.
Suitable Examples:

download from ktuspecial.in

• **Vector processors**.
• **SSE instructions of x86**.
• **Multimedia Extensions** of the x86 architecture, such as Streaming SIMD Extensions (SSE) and Advanced Vector Extensions (AVX). AVX, for instance, supports the simultaneous execution of four 64-bit floating-point numbers.
• The subword parallel instructions seen in Sections 3.6 and 3.7 are also examples of SIMD.
Related Concepts (Vector Architecture):
• An older and more elegant interpretation of SIMD is the **vector architecture**. Vector architectures collect data elements from memory, place them in order into vector registers, operate on them sequentially using pipelined execution units, and write results back to memory.
• Compared to a sequence of scalar instructions, vector instructions specify a great deal of work (equivalent to an entire loop), dramatically reducing instruction fetch and decode bandwidth.
---
3. MIMD (Multiple Instruction streams, Multiple Data streams)
Definition and Characteristics:
• **Instruction/Data Streams:** MIMD stands for **Multiple Instruction streams, Multiple Data streams**.
• **Description:** This architecture represents a conventional **multiprocessor**.
• **Programming Model:** While it is possible to write separate programs for different processors to work together, programmers typically write a single program that runs on all MIMD processors, using conditional statements to direct different processors to different code sections. This common approach is called **Single Program Multiple Data (SPMD)**.
• **Hardware/Software Interface:** MIMD machines can be supported by parallel programming systems like **OpenMP**, which provides an Application Programmer Interface (API) for shared memory multiprocessing. Shared memory MIMD processors are commonly referred to as **multicore** processors today.
Suitable Examples:
• An example of a MIMD architecture is the **Intel Core i7**.
• Multicore computers that use shared memory are typical MIMD systems. Shared memory multiprocessors (SMPs) generally offer a single physical address space across all processors.

What is multithreading in processor design?

Hardware multithreading is a related concept to the Multiple Instruction streams, Multiple Data streams (MIMD) architecture, especially from the programmer's perspective.
Definition and Purpose
Hardware multithreading is defined as increasing the utilization of a processor by **switching to another thread when one thread is stalled**.
It allows **multiple threads to share the functional units of a single processor in an overlapping fashion** to try to utilize the hardware resources efficiently.
In contrast to MIMD, which relies on multiple processes or threads to keep multiple processors busy, hardware multithreading focuses on utilizing the resources of a single processor effectively. To permit this sharing, the processor must duplicate the independent state of each thread. For example, each thread requires a separate copy of the **register file and the program counter**. The memory can be shared using virtual memory mechanisms that already support multi-programming.

download from ktuspecial.in

Crucially, the hardware must support the ability to change threads relatively quickly. A thread switch is intended to be much more efficient (potentially instantaneous) than a process switch, which typically requires hundreds to thousands of processor cycles.

Definition of a Thread

A **thread** is defined as a lightweight process that includes the **program counter, the register state, and the stack**. Threads commonly share a single address space, whereas processes do not.

Main Approaches to Hardware Multithreading

There are two primary traditional approaches to hardware multithreading: fine-grained and coarse-grained.

1. Fine-Grained Multithreading

Fine-grained multithreading involves **switching between threads after every instruction**. This results in the interleaved execution of multiple threads, often in a round-robin fashion, skipping any threads that are stalled during a clock cycle.

• **Advantage:** It can hide throughput losses resulting from both short and long stalls, as instructions from other threads can be executed when one thread stalls.

• **Disadvantage:** It slows down the execution of individual threads, as a thread ready to execute without stalls will be delayed by instructions from other threads.

2. Coarse-Grained Multithreading

Coarse-grained multithreading involves **switching between threads only after significant events**, such as a last-level cache miss.

• **Advantage:** It is much less likely to slow down the execution of an individual thread because instructions from other threads are only issued when a thread encounters a costly stall.

• **Disadvantage:** It is limited in its ability to overcome throughput losses from shorter stalls due to **pipeline start-up costs**. When a stall occurs, the pipeline must be emptied or frozen, and the new thread must refill the pipeline before execution can continue. This overhead means coarse-grained multithreading is most useful for reducing the penalty of high-cost stalls where pipeline refilling is negligible compared to the stall time.

Simultaneous Multithreading (SMT)

Simultaneous multithreading (SMT) is a variation of hardware multithreading that is designed to lower the cost of multithreading by utilizing the resources needed for a **multiple-issue, dynamically scheduled microarchitecture**.

SMT aims to exploit **thread-level parallelism** at the same time it exploits **instruction-level parallelism**.

• **Key Motivation:** Multiple-issue processors often have more functional unit parallelism available than most single threads can effectively use.

• **Mechanism:** SMT takes advantage of register renaming and dynamic scheduling to allow **multiple instructions from independent threads to be issued** without concern for dependencies among them.

• **Execution:** SMT does not switch resources every cycle; instead, it is **always executing instructions from multiple threads**, leaving the hardware to associate instruction slots and renamed registers with their proper threads.

On a single core of the Intel Core i7 960 (which supports two threads via hardware multithreading), the use of multithreading achieved an average speed-up of 1.31 and an average improvement in energy efficiency of 1.07 for the PARSEC benchmarks

download from ktuspecial.in

Describe the role of vector hardware in parallel processing. How does it improve performance?

The role of **vector hardware** in parallel processing is to provide an efficient and elegant interpretation of the **Single Instruction stream, Multiple Data streams (SIMD)** architecture, which is specifically designed to exploit **data-level parallelism**.
Vector architectures have been closely identified with computers designed by Seymour Cray starting in the 1970s.
Role of Vector Hardware in Parallel Processing
Vector hardware, based on the vector architecture model, serves several crucial roles in parallel processing:
1. **Executing SIMD Operations:** Vector architecture is a more elegant interpretation of SIMD. SIMD specifies that the **same instruction is applied to many data streams**. A vector instruction allows an implementation to execute these operations using deeply pipelined functional units, an array of parallel functional units, or a combination of both.
2. **Handling Data-Level Parallelism:** Vector architecture is a great match for problems with **lots of data-level parallelism**. Data-level parallelism is achieved by performing the same operation on independent data.
3. **Managing Data Flow:** The basic philosophy is to **collect data elements from memory, put them in order into a large set of registers (vector registers), operate on them sequentially using pipelined execution units, and then write the results back to memory**. This contrasts with the older "array processors" that simultaneously used multiple ALUs.
4. **Enabling Parallel Operations:** Since vector elements are independent, they can be operated on in parallel. Modern vector computers use vector functional units with **multiple parallel pipelines**, often called **vector lanes**, which can produce two or more results per clock cycle. Adding more lanes increases the peak throughput of a vector unit, similar to adding lanes to a highway.
5. **Supporting Non-Contiguous Memory Access:** Unlike multimedia extensions, vector architectures support flexible data transfers, including **strided accesses** (loading every nth data element) and **indexed accesses**, also known as **gather-scatter** (loading elements based on addresses found in a vector register).
How Vector Hardware Improves Performance
Vector hardware dramatically improves performance compared to conventional scalar architectures and overcomes common bottlenecks in traditional processing:

Performance Improvement Mechanism	Details and Source Support
Reduced Instruction Bandwidth	A single vector instruction specifies a great deal of work, equivalent to executing an entire loop. This dramatically **reduces the instruction fetch and decode bandwidth** required. For example, a benchmark like DAXPY (a loop performing vector multiplication and addition) required about 600

download from ktuspecial.in

	instructions using conventional MIPS code but only **6 instructions** using vector MIPS code. This saving also reduces energy consumption.
Reduced Hazard Checking/Stalls	Hardware does not have to check for data hazards within a vector instruction because the compiler/programmer indicates that the computation of each result is independent of others in the same vector. Hardware only checks for hazards between two *vector instructions* once per vector operand, instead of once for every element within the vectors. Consequently, pipeline stalls are required only once per vector operation, instead of once per vector element (e.g., the stall frequency on conventional MIPS was about 64 times higher than on the vector version for the DAXPY example). Reduced checking also saves energy and time.
Efficient Memory Access	Vector instructions that access memory have a **known access pattern**. If elements are adjacent, fetching the vector from interleaved memory banks works well. Crucially, the cost of latency to main memory is incurred **only once for the entire vector**, rather than once for each word of the vector. Furthermore, vector architectures support **gather-scatter** operations, which are essential for performance when data is scattered in memory (GJK kernel).
Elimination of Control Hazards	Since an entire loop is replaced by a single vector instruction with predetermined behavior, **control hazards** (which normally arise from the loop branch) are nonexistent.
Flexible Implementation and Evolution	Vector architectures are easier to evolve over time than multimedia extensions (like x86 MMX/SSE/AVX). Unlike multimedia extensions where vector length changes require new large sets of opcodes, the number of elements in a vector operation is stored in a separate register in vector architecture. This allows different vector lengths to be implemented just by changing the register content, thus retaining binary compatibility.
Compiler Efficiency	Vector architectures and compilers have a reputation for making it much easier than using MIMD multiprocessors to write efficient applications when the applications contain data-level parallelism.

Discuss the architectural features of Intel Core i7 that support parallel processing.

The Intel Core i7 processor, specifically the Core i7 960 model discussed in the sources, is a conventional multiprocessor (multicore) that supportsThe Intel Core i7 processor, specifically the Core i7 960 model discussed in the sources, is a conventional multiprocessor (multicore) that supports parallel processing primarily through **MIMD architecture, hardware multithreading (SMT), and integrated SIMD instructions (SSE/AVX)**.

Here is a discussion of the architectural features of the Intel Core i7 that support parallel processing, drawing on the provided sources:

1. MIMD (Multiple Instruction, Multiple Data) Architecture

The Core i7 is categorized as a **MIMD** architecture.

download from ktuspecial.in

• **Multiprocessor Classification:** A conventional multiprocessor is defined as having **Multiple Instruction streams, Multiple Data streams**. The Intel Core i7 is explicitly listed as an example of a MIMD architecture.
• **Multicore Design:** Modern MIMD machines are often referred to as **multicore** processors today. The Core i7 960 examined in the sources is a **quad-core** processor, meaning it contains four processing elements (cores).
• **Programming Model (SPMD):** While MIMD allows separate programs on different processors, the normal way to program an MIMD computer is using **Single Program Multiple Data (SPMD)** style. In this style, a single program runs across all processors, using conditional statements to direct different processors to execute different sections of code.
• **Shared Memory:** Multicore chips, including the Core i7, generally offer a single physical address space that all processors can share, which is characteristic of a **shared memory multiprocessor (SMP)**. This is designed so that programs do not need to concern themselves with where their data is located.
• **Parallelization Support (OpenMP):** The shared memory architecture is typically supported by programming systems like **OpenMP**, which provides an Application Programmer Interface (API) for shared memory multiprocessing and is commonly used for parallelizing loops and reductions. For example, optimizations on a dual-chip system with 16 total cores (using Core i7 Sandy Bridge processors) leveraged OpenMP to achieve significant speedups for large matrices.
2. Hardware Multithreading (Simultaneous Multithreading or SMT)
The Intel Core i7 960 supports hardware multithreading, allowing a single core to execute instructions from multiple threads concurrently to improve resource utilization.
• **SMT Mechanism:** The Core i7 uses **Simultaneous Multithreading (SMT)**, a variation of hardware multithreading. SMT exploits **thread-level parallelism** simultaneously with **instruction-level parallelism** by utilizing the resources of a multiple-issue, dynamically scheduled pipelined processor.
• **Resource Sharing:** Hardware multithreading allows multiple threads to share the functional units of a single processor in an overlapping fashion to efficiently utilize hardware resources. The processor must duplicate independent state, such as the **register file and the program counter**, for each thread.
• **Performance Benefits:** Multithreading on a single Core i7 960 core (which supports two threads) achieved an average **speed-up of 1.31** and an average improvement in **energy efficiency of 1.07** for the PARSEC benchmarks, demonstrating the effective use of shared resources.
• **SMT Visualization:** SMT improves issue slot usage by allowing instructions from multiple threads to use issue slots in a single clock cycle, exploiting both thread-level parallelism and instruction-level parallelism, unlike coarse-grained or fine-grained multithreading.
3. SIMD (Single Instruction, Multiple Data) Extensions

download from ktuspecial.in

The Core i7 incorporates SIMD capabilities through multimedia extensions, which enable data-level parallelism by applying a single instruction to multiple data streams.

• **MMX/SSE/AVX Instructions:** The Core i7 960 uses **multimedia SIMD extensions**, specifically **Streaming SIMD Extensions (SSE)** and **Advanced Vector Extensions (AVX)**. The letter 'S' in SSE stands for SIMD.

• **SIMD Operation:** SIMD instructions allow the simultaneous execution of the same instruction across multiple data streams. For instance, AVX supports the simultaneous execution of **four 64-bit floating-point numbers**.

• **Floating-Point Performance:** The Core i7 960 has a **single-precision SIMD width of 4** and a **double-precision SIMD width of 2**. To achieve the highest computational rate (e.g., 51.2 DP GFLOP/sec or 102.4 SP GFLOP/sec), it is necessary to use all four cores and the SSE instructions with an equal number of multiplies and adds.

• **Limitations (Gather-Scatter):** A major limitation of the Core i7's SIMD extensions is the lack of flexible data transfers like **gather-scatter**, which are supported in vector architectures. The multimedia SIMD extensions offer optimal performance only when accesses are aligned on 16-byte boundaries. This lack of efficient gather-scatter support limits performance in kernels where data is scattered in memory (like GJK).

• **Cache Importance:** The Core i7's performance is often heavily reliant on its cache structure. Techniques like **cache blocking** can utilize the caches to prevent memory bandwidth bottlenecks for kernels like Ray Casting, SGEMM, FFT, and SpMV, allowing them to become compute bound.

Explain how shared memory multiprocessors manage data consistency among cores.

Shared memory multiprocessors (SMPs) are designed to offer a programmer a single physical address space across all processors, which is nearly always the case for multicore chips. To manage data consistency in this environment, the architecture relies on specific hardware mechanisms:

1. **Cache Coherence (The Primary Mechanism):** When a common physical address space is used, the **hardware typically provides cache coherence to give a consistent view of the shared memory**. This ensures that all processors see the most up-to-date version of shared data, regardless of which core has cached it.

2. **Memory Consistency Model:** The processor also relies on a **memory consistency model** to ensure correct results. For instance, the Intel Core i7 benefits from such a model, which guarantees the right results even if not all previous accesses to the memory hierarchy have completed.

It is important to note that maintaining data consistency (coherence) is distinct from **synchronization**, which is the process of coordinating the behavior of two or more processes when operating on shared data. Synchronization is typically handled using devices like a **lock**, which ensures that only one processor can access shared data at a time.

download from ktuspecial.in

KTU SPECIAL
CONNECT WITH US

WHATSAPP GROUP

FOLLOW US NOW

TELEGRAM CHANNEL

SUBSCRIBE NOW

www.ktuspecial.in

SUBSCRIBE

FOLLOW US

Visit Website